Executive Brief: Governed Multi-Agent AI Training Curriculum

Objective

Move engineering practice from informal prompting toward auditable, cost-aware agentic workflows. Equip developers to define what an agent may receive, produce, and do, and identify when a human must decide whether work continues.

Business problem

Unstructured AI workflows can expose sensitive context, grant excessive tool access, repeat failed actions, and pass unverified code downstream. Adding agents multiplies handoffs and ownership questions. Engineering teams need explicit acceptance rules, clear responsibilities, and observable stop conditions before extending these workflows.

Workshop approach

A two-day, hands-on workshop teaches governance through bounded repair exercises. Developers validate typed handoffs, scope role-specific context and tools, separate implementation from review, limit retries, and record escalation. The integrated Software Factory adds simulated budget decisions and explicit human gates tied to the candidate revision.

Learners predict a failure, repair a marked control, rerun checks, and explain the evidence. Synthetic data and deterministic fixtures make the supplied cases repeatable without sending learner content to model providers.

Practical outcomes

Participants retain reusable local Python code, repaired notebooks, and a Development-to-Security contract. They inspect actual local files, Git revisions, review records, and publication receipts in a static travel-site exercise. Changed candidates require current evidence and approval. The reference code provides a starting point for further engineering and evaluation.

Evidence and assessment

The portfolio totals **100 marks**: three labs worth 20 each, Capstone implementation worth 20, and a one-page Architecture Decision Record (ADR) and defense worth 20. Lab scores retain their raw results and documented conversion. The ADR explains a concrete risk, observed controls, remaining uncertainty, and a credible next step. Factory and local CI/CD evidence supports the defense without additional marks.

Delivery conditions

Python 3.12, Git, and JupyterLab are required. Initial dependency installation needs internet or a prepared package bundle; workshop execution then uses local fixtures with **no provider calls, API keys, or provider charges**. Actual API cost is **$0.00**; budget figures are simulated teaching values.

The delivered publication is a local file. Hosted deployment, production isolation, privacy compliance, and real-model reliability require additional controls and independent evaluation. Course completion demonstrates the assessed exercises rather than production readiness.